

torch.linalg.eig#

<https://pytorch.org/docs/stable/generated/torch.linalg.eig.html>

Description:

Computes the eigenvalue decomposition of a square matrix if it exists. Letting $K \in \mathbb{R}^n \times \mathbb{R}^n$ or $\mathbb{C}^n \times \mathbb{C}^n$, the eigenvalue decomposition of a square matrix $A \in K^{n \times n}$ (if it exists) is defined as This decomposition exists if and only if A is diagonalizable. This is the case when all its eigenvalues are different. Supports input of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if A is a batch of matrices then the output has the same batch dimensions. The returned eigenvalues are not guaranteed to be in any specific order.

Function Signature

```
torch.linalg.eig(A, *, out=None)#
```

Parameters

Keyword Arguments

Returns

Parameters

Keyword

out (tuple, optional) – output tuple of two tensors. Ignored if None. Default: None.

Returns:

A named tuple (eigenvalues, eigenvectors) which corresponds to Λ and V above. eigenvalues and eigenvectors will always be complex-valued, even when A is real. The eigenvectors will be given by the columns of eigenvectors.

Code Examples

```
>>> A = torch.randn(2, 2, dtype=torch.complex128)
>>> A
tensor([[ 0.9828+0.3889j, -0.4617+0.3010j],
        [ 0.1662-0.7435j, -0.6139+0.0562j]],
        dtype=torch.complex128)
>>> L, V = torch.linalg.eig(A)
>>> L
tensor([ 1.1226+0.5738j, -0.7537-0.1286j],
        dtype=torch.complex128)
>>> V
tensor([[ 0.9218+0.0000j,  0.1882-0.2220j],
        [-0.0270-0.3867j,  0.9567+0.0000j]],
        dtype=torch.complex128)
>>> torch.dist(V @ torch.diag(L) @ torch.linalg.inv(V), A)
tensor(7.7119e-16, dtype=torch.float64)

>>> A = torch.randn(3, 2, 2, dtype=torch.float64)
>>> L, V = torch.linalg.eig(A)
>>> torch.dist(V @ torch.diag_embed(L) @ torch.linalg.inv(V), A)
tensor(3.2841e-16, dtype=torch.float64)
```

Notes & Warnings

NOTE Note The eigenvalues and eigenvectors of a real matrix may be complex.

NOTE

Note When inputs are on a CUDA device, this function synchronizes that device with the CPU.

⚠ WARNING

Warning This function assumes that A is diagonalizable (for example, when all the eigenvalues are different). If it is not diagonalizable, the returned eigenvalues will be correct but $A \neq V \text{diag}(\Lambda) V^{-1}$.

$$A \neq V \text{diag}(\Lambda) V^{-1} \implies A \neq V \text{diag}(\Lambda) V^{-1}$$

⚠ WARNING

Warning The returned eigenvectors are normalized to have norm 1. Even then, the eigenvectors of a matrix are not unique, nor are they continuous with respect to A . Due to this lack of uniqueness, different hardware and software may compute different eigenvectors. This non-uniqueness is caused by the fact that multiplying an eigenvector by $e^{i\phi}$, $\phi \in \mathbb{R}$, produces another set of valid eigenvectors of the matrix. For this reason, the loss function shall not depend on the phase of the eigenvectors, as this quantity is not well-defined. This is checked when computing the gradients of this function. As such, when inputs are on a CUDA device, the computation of the gradients of this function synchronizes that device with the CPU.

⚠ WARNING

Warning Gradients computed using the eigenvectors tensor will only be finite when A has distinct eigenvalues. Furthermore, if the distance between any two eigenvalues is close to zero, the gradient will be numerically unstable, as it depends on the eigenvalues λ_i through the computation of $\frac{1}{\min_{i \neq j} |\lambda_i - \lambda_j|}$.

NOTE

See also `torch.linalg.eigvals()` computes only the eigenvalues. Unlike `torch.linalg.eig()`, the gradients of `eigvals()` are always numerically stable. `torch.linalg.eigh()` for a (faster) function that computes the eigenvalue decomposition for Hermitian and symmetric matrices. `torch.linalg.svd()` for a function that computes another type of spectral decomposition that works on matrices of any shape. `torch.linalg.qr()` for another (much faster) decomposition that works on matrices of any shape.

Detailed Documentation

Documentation

Computes the eigenvalue decomposition of a square matrix if it exists.

Letting $K \in \mathbb{R}^{n \times n}$ or $\mathbb{C}^{n \times n}$, the eigenvalue decomposition of a square matrix $A \in \mathbb{K}^{n \times n}$ (if it exists) is defined as

This decomposition exists if and only if A is diagonalizable. This is the case when all

its eigenvalues are different.

Supports input of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if A is a batch of matrices then the output has the same batch dimensions.

The returned eigenvalues are not guaranteed to be in any specific order.

The eigenvalues and eigenvectors of a real matrix may be complex.

When inputs are on a CUDA device, this function synchronizes that device with the CPU.

This function assumes that A is diagonalizable (for example, when all the eigenvalues are different). If it is not diagonalizable, the returned eigenvalues will be correct but $A \neq V \text{diag}(\Lambda) V^{-1}$. $A \neq V \text{diag}(\Lambda) V^{-1}$.

The returned eigenvectors are normalized to have norm 1. Even then, the eigenvectors of a matrix are not unique, nor are they continuous with respect to A . Due to this lack of uniqueness, different hardware and software may compute different eigenvectors.

This non-uniqueness is caused by the fact that multiplying an eigenvector by $e^{i\phi}$, $\phi \in \mathbb{R}$ produces another set of valid eigenvectors of the matrix. For this reason, the loss function shall not depend on the phase of the eigenvectors, as this quantity is not well-defined. This is checked when computing the gradients of this function. As such, when inputs are on a CUDA device, the computation of the gradients of this function synchronizes that device with the CPU.

Gradients computed using the eigenvectors tensor will only be finite when A has distinct eigenvalues. Furthermore, if the distance between any two eigenvalues is close to zero, the gradient will be numerically unstable, as it depends on the eigenvalues

λ_i through the computation of $\frac{1}{\min_{i \neq j} |\lambda_i - \lambda_j|}$.

`torch.linalg.eigvals()` computes only the eigenvalues. Unlike `torch.linalg.eig()`, the gradients of `eigvals()` are always numerically stable.

`torch.linalg.eigh()` for a (faster) function that computes the eigenvalue decomposition for Hermitian and symmetric matrices.

`torch.linalg.svd()` for a function that computes another type of spectral decomposition that works on matrices of any shape.

`torch.linalg.qr()` for another (much faster) decomposition that works on matrices of any shape.

A (Tensor) – tensor of shape $(*, n, n)$ where $*$ is zero or more batch dimensions consisting of diagonalizable matrices.

out (tuple, optional) – output tuple of two tensors. Ignored if None. Default: None.

A named tuple (eigenvalues, eigenvectors) which corresponds to Λ and V above.

eigenvalues and eigenvectors will always be complex-valued, even when A is real. The eigenvectors will be given by the columns of eigenvectors.